

Name : Kasak Patel

Roll No : 05

Sec-Batch : B-B1

ML Lab 2

Lab-2 : To develop a Linear Regression model for predicting house/car prices using suitable dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
```

df.head(): Displays the first 5 rows of the dataset by default. It is used to quickly preview the data.

```
df=pd.read_csv('USA_Housing.csv')
df.head()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	\
0	79545.458574	5.682861	7.009188	
1	79248.642455	6.002900	6.730821	
2	61287.067179	5.865890	8.512727	
3	63345.240046	7.188236	5.586729	
4	59982.197226	5.040555	7.839388	

	Avg. Area Number of Bedrooms	Area Population	Price	\
0	4.09	23086.800503	1.059034e+06	
1	3.09	40173.072174	1.505891e+06	
2	5.13	36882.159400	1.058988e+06	
3	3.26	34310.242831	1.260617e+06	
4	4.23	26354.109472	6.309435e+05	

	Address
0	208 Michael Ferry Apt. 674\nLaurabury, NE 3701...
1	188 Johnson Views Suite 079\nLake Kathleen, CA...
2	9127 Elizabeth Stravenue\nDanielstown, WI 06482...
3	USS Barnett\nFPO AP 44820
4	USNS Raymond\nFPO AE 09386

df.tail(): Displays the last 5 rows of the dataset by default. It is used to view the ending records of the data.

```
df.tail()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	
4995	60567.944140	7.830362	6.137356	
4996	78491.275435	6.999135	6.576763	
4997	63390.686886	7.250591	4.805081	
4998	68001.331235	5.534388	7.130144	
4999	65510.581804	5.992305	6.792336	

	Avg. Area Number of Bedrooms	Area Population	Price	
4995	3.46	22837.361035	1.060194e+06	
4996	4.02	25616.115489	1.482618e+06	
4997	2.13	33266.145490	1.030730e+06	
4998	5.44	42625.620156	1.198657e+06	
4999	4.07	46501.283803	1.298950e+06	

	Address
4995	USNS Williams\nFP0 AP 30153-7653
4996	PSC 9258, Box 8489\nAP0 AA 42991-3352
4997	4215 Tracy Garden Suite 076\nJoshualand, VA 01...
4998	USS Wallace\nFP0 AE 73316
4999	37778 George Ridges Apt. 509\nEast Holly, NV 2...

df.shape: Returns the number of rows and columns in the dataset.

```
df.shape
```

```
(5000, 7)
```

df.info(): Displays a summary of the dataset, including the number of rows, columns, data types, and missing values.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Avg. Area Income                      5000 non-null   float64
1   Avg. Area House Age                   5000 non-null   float64
2   Avg. Area Number of Rooms             5000 non-null   float64
3   Avg. Area Number of Bedrooms          5000 non-null   float64
```

4	Area Population	5000 non-null	float64
5	Price	5000 non-null	float64
6	Address	5000 non-null	object

dtypes: float64(6), object(1)
memory usage: 273.6+ KB

df.isnull().sum(): Displays the total number of missing (null) values in each column of the dataset.

```
df.isnull().sum()
```

Avg. Area Income	0
Avg. Area House Age	0
Avg. Area Number of Rooms	0
Avg. Area Number of Bedrooms	0
Area Population	0
Price	0
Address	0

dtype: int64

df.describe(): Displays the statistical summary of numerical columns, such as count, mean, standard deviation, minimum, maximum, and quartiles.

```
df.describe()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms
count	5000.000000	5000.000000	5000.000000
mean	68583.108984	5.977222	6.987792
std	10657.991214	0.991456	1.005833
min	17796.631190	2.644304	3.236194
25%	61480.562388	5.322283	6.299250
50%	68804.286404	5.970429	7.002902
75%	75783.338666	6.650808	7.665871
max	107701.748378	9.519088	10.759588

	Avg. Area Number of Bedrooms	Area Population	Price
count	5000.000000	5000.000000	5.000000e+03
mean	3.981330	36163.516039	1.232073e+06
std	1.234137	9925.650114	3.531176e+05
min	2.000000	172.610686	1.593866e+04
25%	3.140000	29403.928702	9.975771e+05

50%	4.050000	36199.406689	1.232669e+06
75%	4.490000	42861.290769	1.471210e+06
max	6.500000	69621.713378	2.469066e+06

```
pd.set_option('display.float_format', '(:,2f)'.format)
```

```
df.head()
```

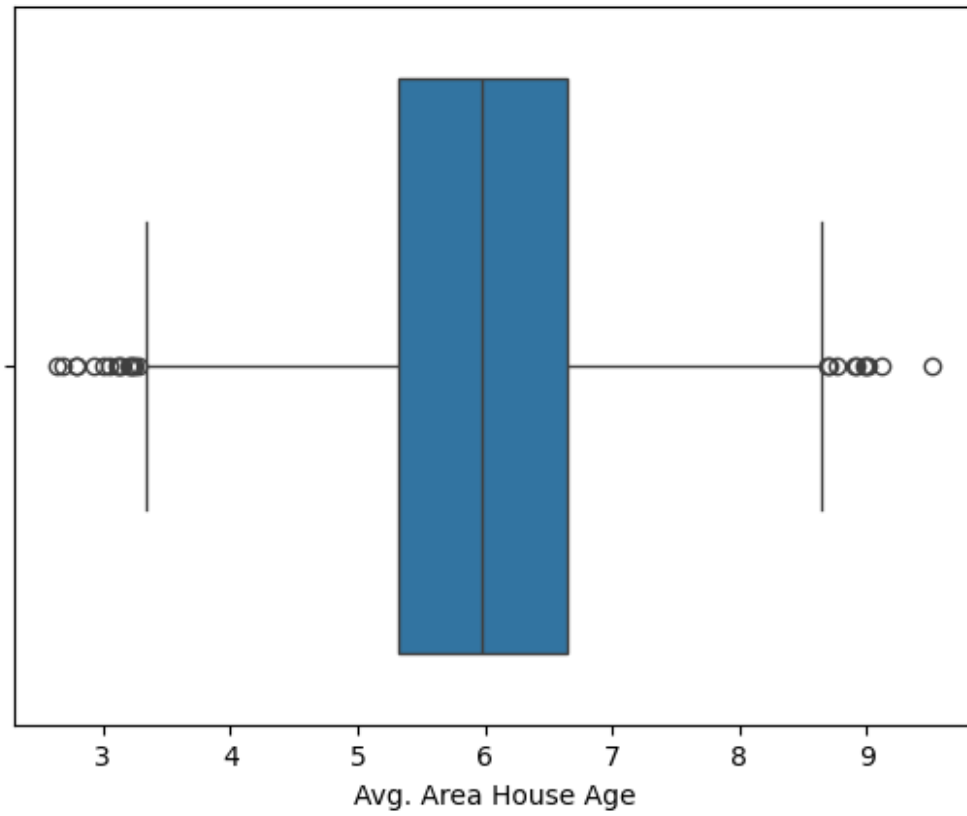
	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	\
0	(:,2f)	(:,2f)	(:,2f)	
1	(:,2f)	(:,2f)	(:,2f)	
2	(:,2f)	(:,2f)	(:,2f)	
3	(:,2f)	(:,2f)	(:,2f)	
4	(:,2f)	(:,2f)	(:,2f)	

	Avg. Area Number of Bedrooms	Area Population	Price	\
0	(:,2f)	(:,2f)	(:,2f)	
1	(:,2f)	(:,2f)	(:,2f)	
2	(:,2f)	(:,2f)	(:,2f)	
3	(:,2f)	(:,2f)	(:,2f)	
4	(:,2f)	(:,2f)	(:,2f)	

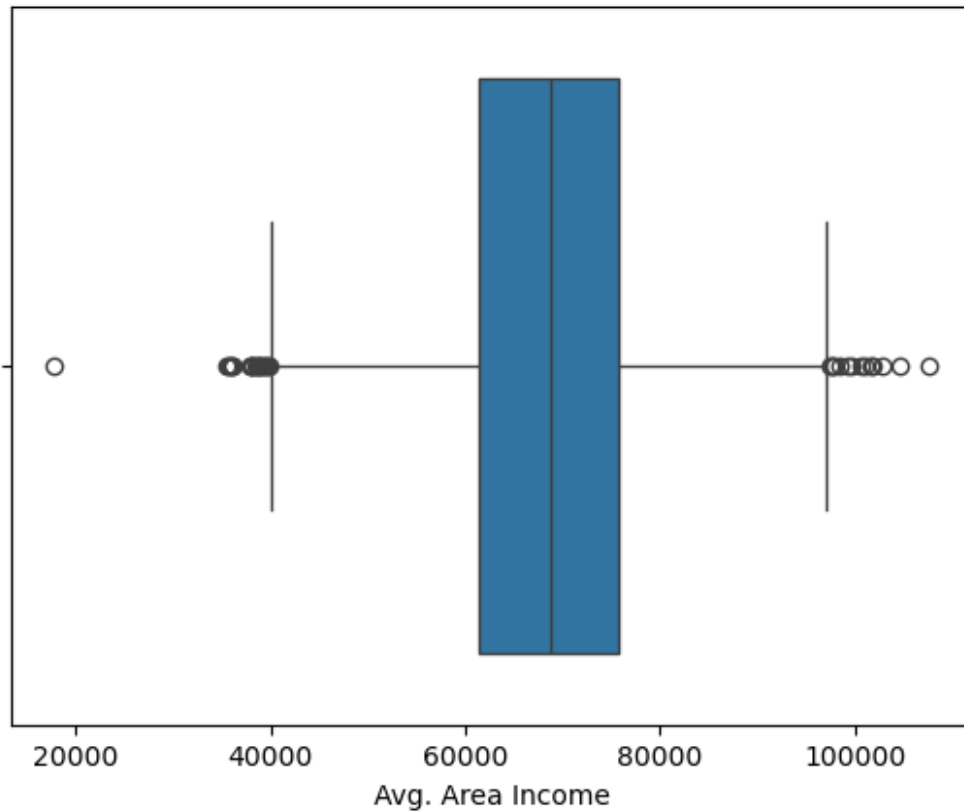
	Address
0	208 Michael Ferry Apt. 674\nLaurabury, NE 3701...
1	188 Johnson Views Suite 079\nLake Kathleen, CA...
2	9127 Elizabeth Stravenue\nDanielstown, WI 06482...
3	USS Barnett\nFP0 AP 44820
4	USNS Raymond\nFP0 AE 09386

`sns.boxplot(x=df["Avg. Area House Age"])`: Creates a box plot to visualize the distribution of the Avg. Area House Age feature and identify any outliers.

```
sns.boxplot(x=df["Avg. Area House Age"])
plt.show()
```

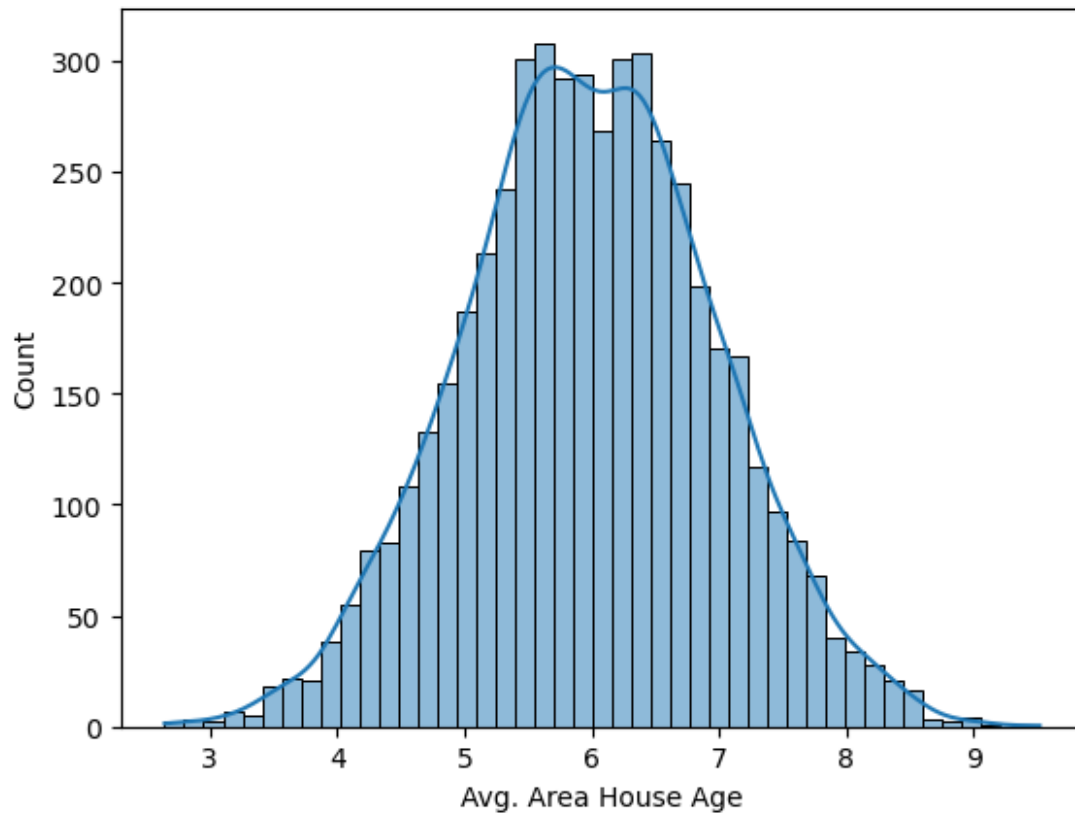


```
sns.boxplot(x=df["Avg. Area Income"])\nplt.show()
```

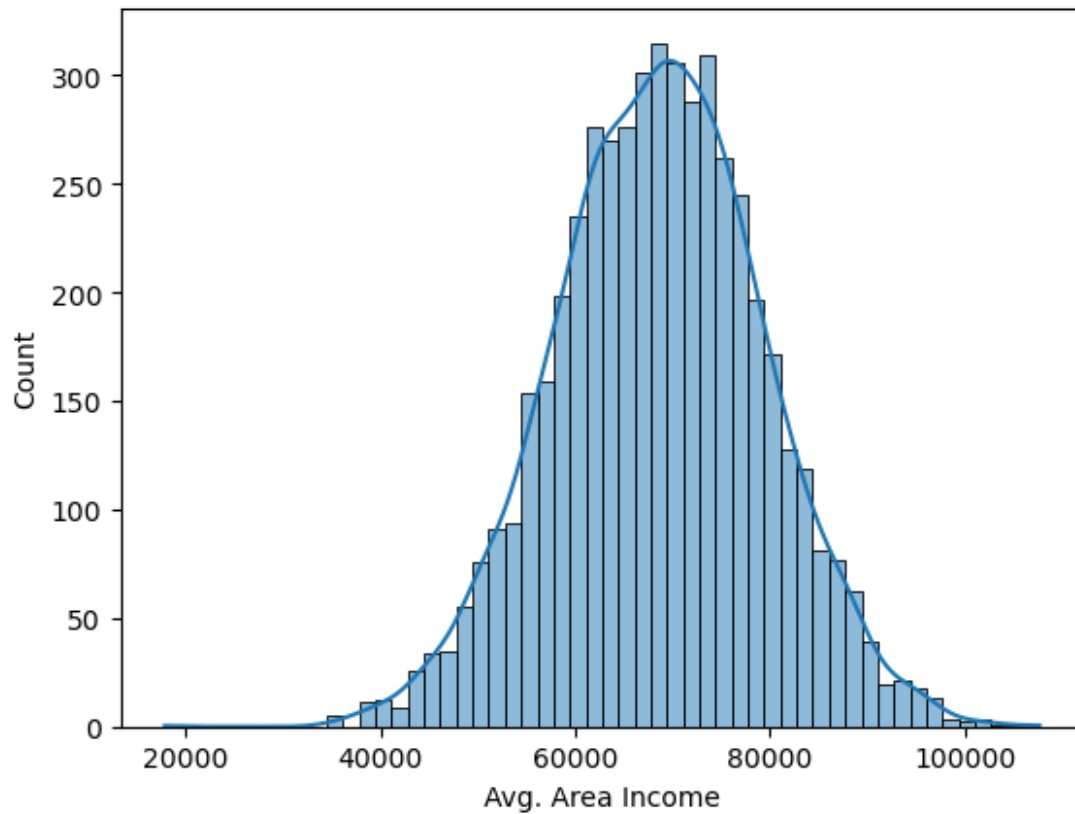


`sns.histplot(df["Avg. Area House Age"], kde=True)`: Plots a histogram with a Kernel Density Estimate (KDE) curve to show the distribution of the Avg. Area House Age feature.

```
sns.histplot(df["Avg. Area House Age"], kde=True) # Kernel Density Estimate
plt.show()
```



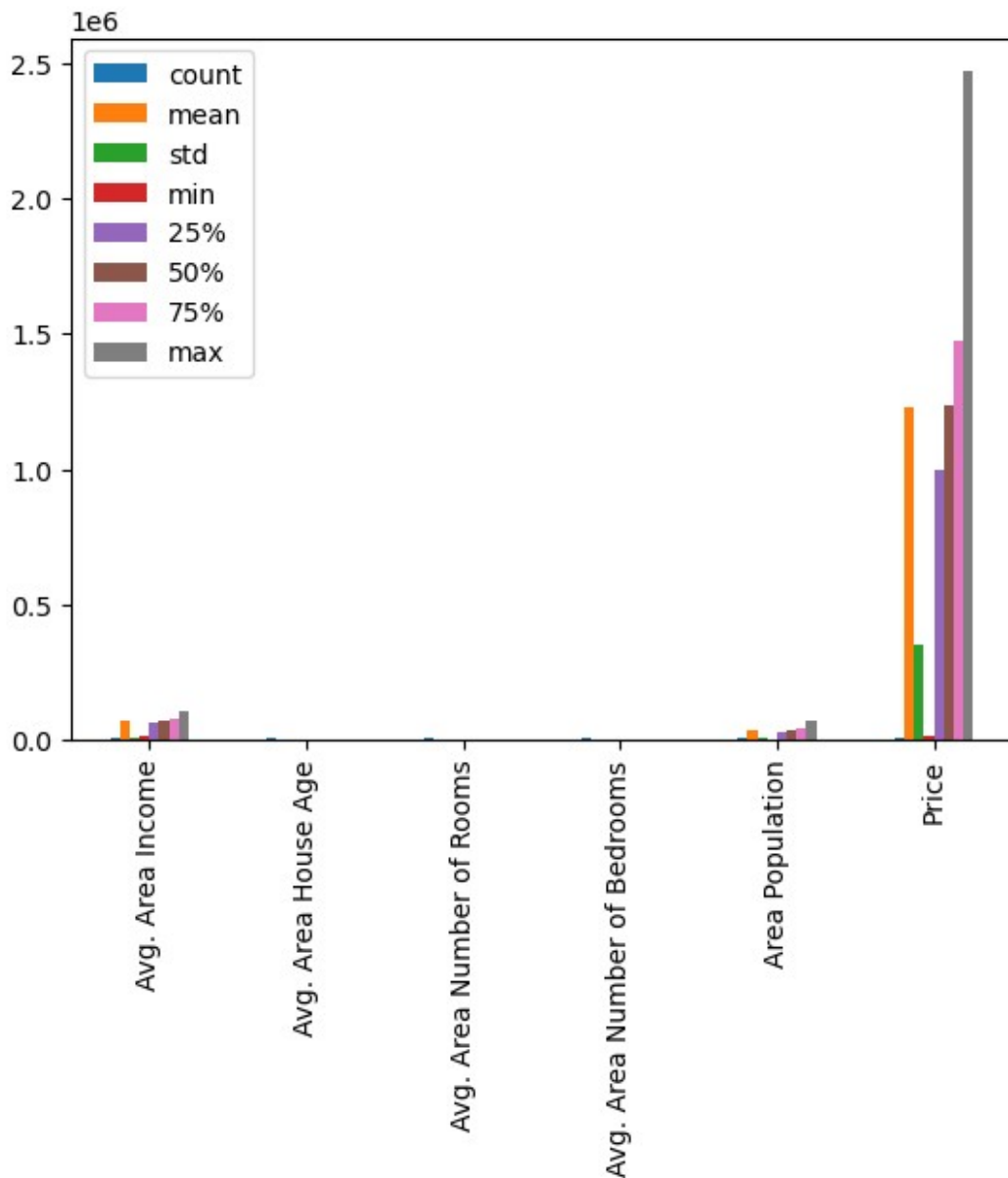
```
sns.histplot(df["Avg. Area Income"], kde=True)  
plt.show()
```



`df.describe().T.plot(kind='bar')`: Creates a bar chart of the statistical summary (count, mean, standard deviation, etc.) for all numerical columns in the dataset.

```
df.describe().T.plot(kind='bar')
```

<Axes: >



`numeric_df = df.select_dtypes(include=['number'])`: Selects only the numerical columns from the dataset.

`sns.heatmap(numeric_df.corr(), annot=True, cmap='inferno')`: Creates a correlation heatmap showing the relationship between numerical features. The `annot=True` option displays the correlation values, and `cmap='inferno'` sets the color theme.

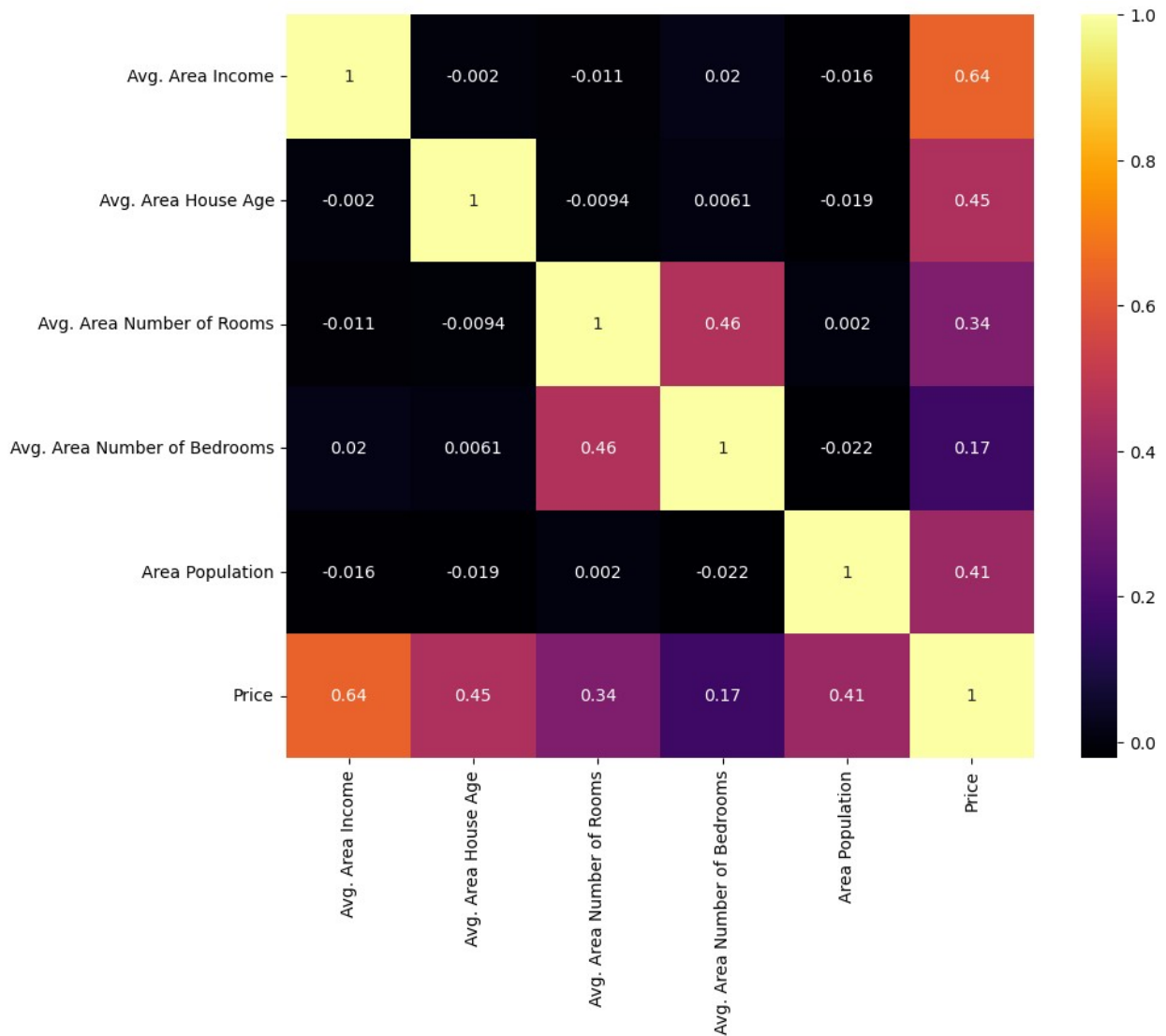
multivariate analysis

```
plt.figure(figsize=(10, 8))
numeric_df = df.select_dtypes(include=['number'])
sns.heatmap(numeric_df.corr(),
```

```

        annot=True,
        cmap='inferno')
plt.show()

```



SLR

```

x = df[['Avg. Area Income']]
y = df['Price']

```

`train_test_split()`: Splits the dataset into training (80%) and testing (20%) sets. The `random_state=0` ensures the split is reproducible every time the code is run.

```

from sklearn.model_selection import train_test_split

```

```
X_train, X_test, y_train, y_test = train_test_split(x, y,
test_size=0.2, random_state=0)
```

LinearRegression(): Creates a Linear Regression model.

slr_model.fit(X_train, y_train): Trains the Linear Regression model using the training data (X_train and y_train).

```
from sklearn.linear_model import LinearRegression

slr_model = LinearRegression()

slr_model.fit(X_train, y_train)

LinearRegression()
```

slr_model.predict(X_test): Predicts the target values for the test data using the trained Linear Regression model. The predicted values are stored in y_pred.

```
y_pred = slr_model.predict(X_test)
```

mean_absolute_error(y_test, y_pred): Calculates the Mean Absolute Error (MAE), which measures the average prediction error.

mean_squared_error(y_test, y_pred): Calculates the Mean Squared Error (MSE), which measures the average squared prediction error.

np.sqrt(mean_squared_error(y_test, y_pred)): Calculates the Root Mean Squared Error (RMSE), representing the prediction error in the original units.

r2_score(y_test, y_pred): Calculates the R² Score, which indicates how well the model fits the data.

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
import numpy as np

print("Simple Linear Regression Results")
print()
print("MAE :", mean_absolute_error(y_test, y_pred))
print("MSE :", mean_squared_error(y_test, y_pred))
print("RMSE :", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R^2 :", r2_score(y_test, y_pred))
```

Simple Linear Regression Results

```
MAE : 219728.33602368194
MSE : 74084394125.46416
```

```
RMSE : 272184.4854606231  
R^2 : 0.40060598064566877
```

```
print("Intercept:", slr_model.intercept_)  
print("Slope:", slr_model.coef_[0])
```

```
Intercept: -224617.8010927781  
Slope: 21.243682487413853
```

```
plt.figure(figsize=(8,6)) # Creates a figure of size 8 × 6 inches.
```

```
plt.scatter(X_test,y_test,color='blue',label='Actual Data') # Plots  
the actual test data as blue scatter points.
```

```
plt.plot(X_test,y_pred,color='red',linewidth=2,label='Regression  
Line') # Draws the Linear Regression prediction line in red.
```

```
plt.xlabel("Average Area Income") # Sets the x-axis label.
```

```
plt.ylabel("Housing Price") # Sets the y-axis label.
```

```
plt.title("Sample Linear Regression") # Adds a title to the graph.
```

```
plt.legend() # Displays the legend for the graph.
```

```
plt.show()
```



```
income = float(input("Enter Average Area Income: "))
new_data = np.array([[income]])
prediction = slr_model.predict(new_data)
print("Predicted House Price = ${:,.2f}".format(prediction[0]))
```

Enter Average Area Income: 453453

Predicted House Price = \$9408393.75

```
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\base.py:493:
UserWarning: X does not have valid feature names, but LinearRegression
was fitted with feature names
warnings.warn(
```

Ridge Regression (Hyperparameter Tuning)

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge
ridge=Ridge()
```

```

ridge.fit(X_train,y_train)
ridge_pred=ridge.predict(X_test)
print("default ridge r^2 : ",r2_score(y_test,ridge_pred))

default ridge r^2 : 0.4006059806456894

```

param_grid: Defines different alpha values to test and find the best regularization strength for the Ridge Regression model.

```

param_grid={'alpha': [0.001,0.01,0.1,1,10,100]}

```

GridSearchCV(): Performs hyperparameter tuning to find the best Ridge Regression model by testing different parameter values using 5-fold cross-validation and selecting the model with the highest R² score.

```

grid_ridge=GridSearchCV(estimator=Ridge(),param_grid=param_grid,scoring='r2',cv=5)

grid_ridge.fit(X_train,y_train)

GridSearchCV(cv=5, estimator=Ridge(),
              param_grid={'alpha': [0.001, 0.01, 0.1, 1, 10, 100]},
              scoring='r2')

print("best alpha:",grid_ridge.best_params_)

best alpha: {'alpha': 100}

print(grid_ridge.best_score_)

0.4092374970280138

```

Evaluates the best Ridge Regression model on the test dataset using MAE, MSE, RMSE, and R² Score.

```

best_ridge = grid_ridge.best_estimator_
ridge_pred = best_ridge.predict(X_train)
best_ridge = grid_ridge.best_estimator_

# Predict on test set (not training)
ridge_pred = best_ridge.predict(X_test)

print("MAE :", mean_absolute_error(y_test, ridge_pred))
print("MSE :", mean_squared_error(y_test, ridge_pred))
print("RMSE :", np.sqrt(mean_squared_error(y_test, ridge_pred)))
print("R^2 :", r2_score(y_test, ridge_pred))

MAE : 219728.33602273924
MSE : 74084394125.20862
RMSE : 272184.4854601537
R^2 : 0.4006059806477362

```

Lasso Regression

Finds the best Lasso Regression model by tuning the alpha value and evaluates its performance on the test data using the R² Score.

```
from sklearn.linear_model import Lasso
param_grid = {'alpha':[0.001,0.01,0.1,1,10]}

grid_lasso =
GridSearchCV(Lasso(max_iter=5000),param_grid,cv=5,scoring='r2')
grid_lasso.fit(X_train,y_train)

print(grid_lasso.best_params_)

lasso_pred = grid_lasso.predict(X_test)
print(r2_score(y_test,lasso_pred))

{'alpha': 10}
0.40060598068459463
```

MLR (Multiple Linear Regression)

Separates the dataset into input features (X) by removing Price and Address, and the target variable (y) as Price.

```
X=df.drop(["Price","Address"],axis=1)
y=df["Price"]
```

Splits the dataset into 80% training data and 20% testing data for model training and evaluation.

```
X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42)

# Creates and trains a Multiple Linear Regression model using the training data.
mlr=LinearRegression()
mlr.fit(X_train,y_train)

LinearRegression()

# Uses the trained Multiple Linear Regression model to predict house prices for the test data.
y_pred=mlr.predict(X_test)

print("MAE :", mean_absolute_error(y_test, ridge_pred))
print("MSE :", mean_squared_error(y_test, ridge_pred))
```

```

print("RMSE :", np.sqrt(mean_squared_error(y_test, ridge_pred)))
print("R^2 :", r2_score(y_test, ridge_pred))

MAE : 334562.1354779434
MSE : 181380069786.52826
RMSE : 425887.3909691719
R^2 : -0.47424573221611555

plt.figure(figsize=(6,6))

<Figure size 600x600 with 0 Axes>

<Figure size 600x600 with 0 Axes>

# Predicts the house prices for the test dataset using the trained
Multiple Linear Regression model and stores them in predictions.
predictions = mlr.predict(X_test)

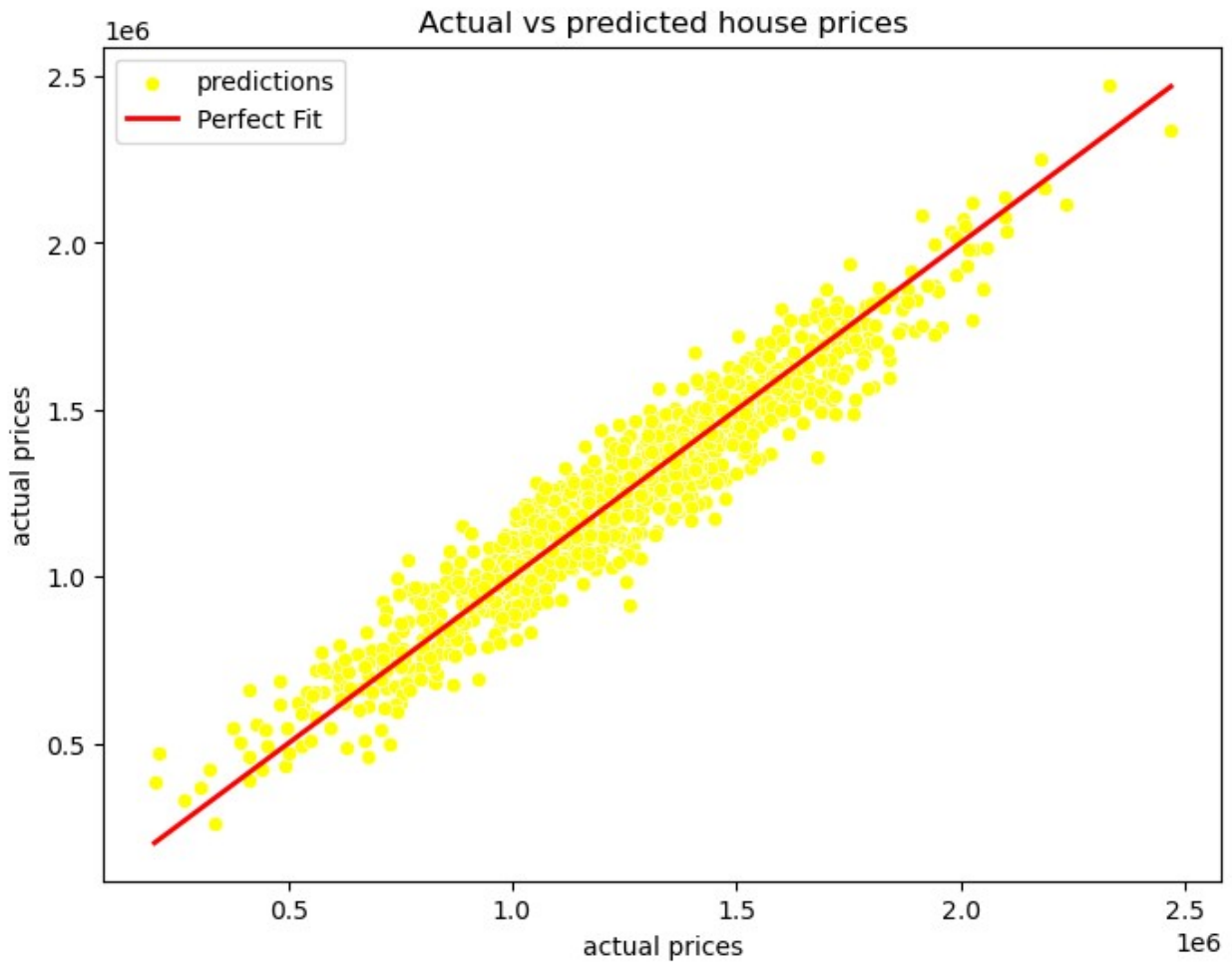
```

Creates a scatter plot comparing actual house prices with predicted house prices and shows the Perfect Fit line for model evaluation.

```

plt.figure(figsize=(8,6))
sns.scatterplot(x=y_test,y=predictions,color='yellow',label='predictions')
plt.plot([min(y_test),max(y_test)],
[min(y_test),max(y_test)],color='red',linewidth=2,label='Perfect Fit')
plt.title('Actual vs predicted house prices')
plt.xlabel('actual prices')
plt.ylabel('actual prices')
plt.legend()
plt.show()

```

```
#Take input from the user
income = float(input("Enter Average Area Income: "))
house_age = float(input("Enter Average Area House Age: "))
rooms = float(input("Enter Average Area Number of Rooms: "))
bedrooms = float(input("Enter Average Area Number of Bedrooms: "))
population = float(input("Enter Area Population: "))

#Creat input array
test_input = np.array([[income, house_age, rooms, bedrooms,
population]])

#Predict price
predicted_price = mlr.predict(test_input)

#Display result
print("\nPredicted House Price: ₹{:, .2f}".format(predicted_price[0]))

Enter Average Area Income: 4534534
Enter Average Area House Age: 19
Enter Average Area Number of Rooms: 25
```

```
Enter Average Area Number of Bedrooms: 12
Enter Area Population: 354353
```

```
Predicted House Price: ₹107,107,219.65
```

```
C:\Users\HP\anaconda3\Lib\site-packages\sklearn\base.py:493:
UserWarning: X does not have valid feature names, but LinearRegression
was fitted with feature names
  warnings.warn(
```

Viva Questions with Answers

1. What is Linear Regression?

Answer: A supervised learning algorithm used to predict continuous values.

1. What is the difference between Simple and Multiple Linear Regression?

Answer: Simple uses one feature; Multiple uses two or more features.

1. Why do we use the Train-Test Split?

Answer: To train the model and test its accuracy on new data.

1. What is the R^2 Score?

Answer: It measures how well the model fits the data. Higher is better.

1. What is MAE?

Answer: Mean Absolute Error (MAE) is the average difference between actual and predicted values. Lower MAE means better accuracy.